Studio Nx

JS 271 — 저장 · 자동 저장 · 떠나기 경고

2026-10-19

JavaScript 271 — 저장 · 자동 저장 · 떠나기 경고 (`sequencer-save.js`)

항목	내용
파일	`studionx/frontend/static/js/pages/sequencer-save.js`
작업	S1-10 프로젝트 전체 저장·불러오기: 수동 저장 · 자동 저장 · 떠나기 경고 (PRD 02 FR-253, FR-256)
작성	Claude (AI), 2026-10-19, 사용자 검토
이전 · 다음	270 되돌리기 · 다시 하기 · 272 시퀀서 시작과 전체 불러오기

1. 하는 일

BPM · Key 같은 곡 설정이 바뀌면 "저장할 것"으로 모아 두었다가 Ctrl+S 나 자동 저장 때 한 번에 보냅니다. 저장 안 된 것이 남아 있으면 창을 닫으려 할 때 브라우저가 한 번 물어봅니다.

```
queueSettings({ bpm: 140 })   → pending = { bpm: 140 }, dirty = true
        ↓
   30초 지나면(saveNow('auto')) 또는 Ctrl+S(saveNow('menu'))
        ↓
PATCH /projects/{id}/settings  ← pending 만 담아 보냅니다
        ↓
  성공 → pending 비우고 #save-state 칩을 ● Saved 로
  실패 → dirty 유지, 다시 시도
```

2. 언제 쓰나

쓰는 곳	
`sequencer.js` `handleChange`	`bpm` · `key` 변화를 `queueSettings` 로 넘깁니다
메뉴 FILE > Save	`project.save` 명령이 `saveNow('menu')` 를 부릅니다
Ctrl+S	`initShortcuts` 가 같은 `project.save` 명령을 실행합니다
자동 저장	마지막 변화 후 30초 뒤 `saveNow('auto')` 를 실행합니다
떠나기	`beforeunload` 이벤트가 저장 안 된 것이 있으면 경고를 띄웁니다

3. 코드

```
export const SETTINGS_KEYS = ['bpm', 'time_sig_num', 'time_sig_den', 'music_key', 'scale_mode', 'sample_
rate', 'master_settings'];
export const AUTOSAVE_MS = 30000;
let projectId = null; let dirty = false; let pending = {}; let saving = false;
let timer = 0; let autosaveMs = AUTOSAVE_MS; let toast = function () {};

export function initSave(options) {
  const opts = options || {};
  if (typeof opts.toast === 'function') toast = opts.toast;
```

```
  if (opts.autosaveMs) autosaveMs = opts.autosaveMs;
  if (opts.projectId !== undefined) setProjectId(opts.projectId);
  registerCommand('project.save', function () { saveNow('menu'); }, { label: 'Save project', group: 'fil
e' });
  registerCommand('project.discard', discardChanges, { label: 'Discard unsaved changes', group: 'file'
});
  window.addEventListener('beforeunload', onBeforeUnload);
  paintState();
}

export function queueSettings(patch) {
  let added = false;
  SETTINGS_KEYS.forEach(function (key) {
    if (patch && Object.prototype.hasOwnProperty.call(patch, key)) {
      pending[key] = patch[key];
      added = true;
    }
  });
  if (added) markDirty('settings');
}

export async function saveNow(reason) {
  if (!projectId) { if (reason === 'menu') toast('Saving needs the server'); return false; }
  if (saving) return false;
  if (!dirty) { if (reason === 'menu') toast('Everything is saved'); return true; }
  const body = Object.assign({}, pending);
  saving = true;
  try {
    await api.patch('/projects/' + encodeURIComponent(projectId) + '/settings', body);
    Object.keys(body).forEach(function (key) {
      if (pending[key] === body[key]) delete pending[key];
    });
    dirty = Object.keys(pending).length > 0;
    saving = false;
    paintState();
    if (reason === 'menu') toast('Saved');
    return true;
  } catch (err) {
    saving = false; dirty = true; paintState();
    toast(err && err.status === 403 ? 'You cannot save this project' : 'Could not save — we will try aga
in');
    if (!timer) timer = window.setTimeout(function () { timer = 0; saveNow('auto'); }, autosaveMs);
    return false;
  }
}

function onBeforeUnload(event) {
  if (!dirty) return undefined;
  event.preventDefault();
  event.returnValue = '';
  return '';
}
```

문법 풀이

식	뜻
`SETTINGS_KEYS`	서버가 받는 칸 이름 목록. 이 칸만 pending 에 넣습니다
`Object.prototype.hasOwnProperty.call(patch, key)`	`patch` 에 실제로 들어 있는 칸만 병합합니다
`Object.keys(body).forEach` + `pending[key] === body[key]`	저장하는 동안 다른 변화가 오면 그 칸은 남깁니다
`event.preventDefault()` + `returnValue = ''`	브라우저가 "정말 떠나시겠습니까?" 를 띄웁니다
`window.setTimeout(..., autosaveMs)`	마지막 변화 후 30초를 기다립니다. 중간에 다른 변화가 오면 `markDirty` 가 다시 예약합니다

4. 왜 이렇게

선택	이유
pending 모아서 한번에	노브를 빠르게 돌릴 때마다 PATCH 를 보내면 서버가 힘듭니다
30초 자동 저장	FR-253 이 자동 저장을 요구합니다. 매번 즉시 하지 않아도 되고, 오래 기다리지도 않습니다
`saving` 깃발	이미 저장 중일 때 다시 누르면 겹쳐서 보내지 않습니다
`beforeunload` 경고	FR-256. 저장 안 된 것이 있으면 브라우저 기본 확인 창을 띄웁니다
`paintState()`	상단 칩(#save-state)을 ● Saved / ● Unsaved changes / ● Saving… 으로 바꿉니다

5. 시험

`docs/handoff/s1-10_tests/ui.mjs` 에서 BPM 칩을 바꾼 뒤 Ctrl+S 를 누르면 `• Saved` 가 되고 `PATCH /projects/{id}/settings` 가 갔는지, 자동 저장이 30초 안에 일어나는지(가짜 타이머로 확인), `beforeunload` 가 저장 안 된 상태에서만 경고를 띄우는지 확인합니다.